Wiktor Morawiec
Radosław Blacha

C++ RANGES

Revision: Lambda Functions

```
#include <iostream>

int main() {

    auto lambda = [](int x) { return x + 1; };
    std::cout << lambda(1) << std::endl;

    return 0;
}
```

Predefined lambda function

Used in scenarios where the lambda function will be invoked later in the code.

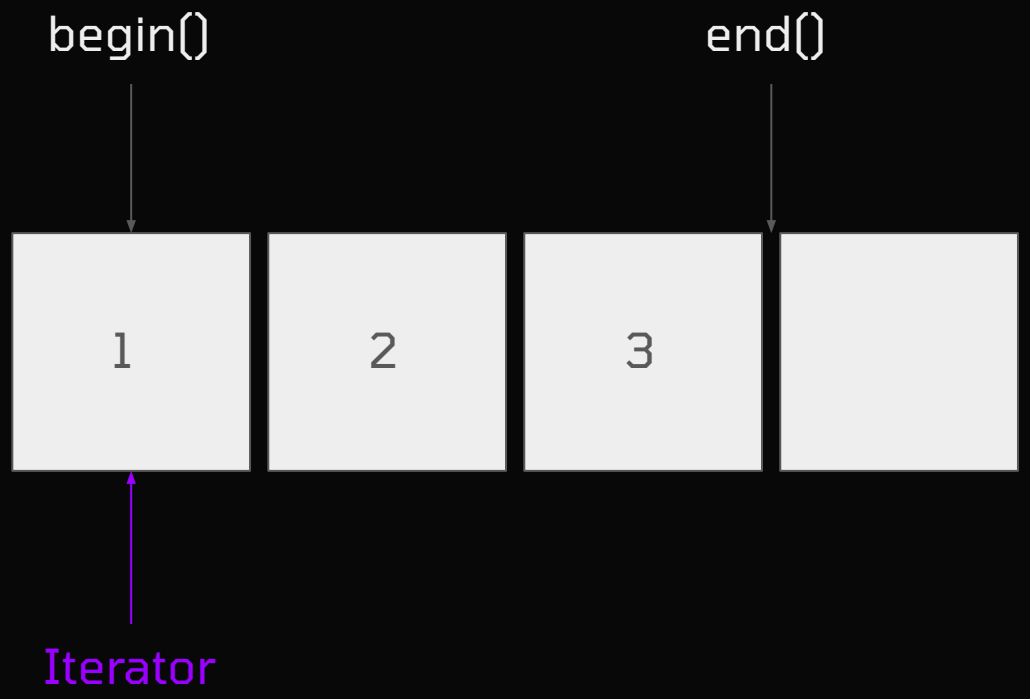
```
#include <iostream>

int main() {

    int i = 1;
    [i]() { std::cout << i << std::endl; }();

    return 0;
}
```

Lambda function with a capture

Takes a value from outside its scope, is invoked with the () at the end of the line.

What is a range?

Range is an iterable sequence, in case of regular STL containers examples include std::vector, std::array, std::map and std::list.




```
#include <vector>

int main() {

    std::vector sampleVector = { 1, 2, 3 };

    return 0;
}
```

The <ranges> library

1_

More readable

With the use of the pipeline all filtering and transformations can be done in a single step.

2_

Less error-prone

Treats the range as a single object.

3_

More efficient

No need for intermediate containers, avoiding unnecessary memory allocation.

Without <ranges>:

```cpp
int main() {

    std::vector<int> sampleVector = { 1, 2, 3, 4 };
    std::vector<int> vector2, vector3;

    std::copy_if(sampleVector.begin(), sampleVector.end(), std::back_inserter(vector2), [](int i) { return i % 2 == 0; });
    std::transform(vector2.begin(), vector2.end(), std::back_inserter(vector3), [](int i) { return i + 10; });

    for (int x : vector3) {
        std::cout << x << std::endl;
    }

    return 0;
}
```

With <ranges>:

```cpp
int main() {

    std::vector<int> sampleVector = { 1, 2, 3, 4 };
    auto even = [](int x) { return x % 2 == 0; };
    auto inc10 = [](int x) { return x + 10; };

    auto output = sampleVector
        | std::views::filter(even)
        | std::views::transform(inc10);

    for (int x : output) {
        std::cout << x << std::endl;
    }

    return 0;
}
```

Microsoft Visual Studio Debu

```
12
14
```

std::ranges::views:: ...	
... filter	Discards all elements that do not meet a specified requirement.
... transform	Applies a specific transformation function to each of the elements.
... take	Keeps first N elements of a range.
... drop	Discards first N elements of a range.
... reverse	Iterates the next condition from the end instead of the beginning of the range.
... join	Combines smaller ranges.
... split	Divides a range into smaller ones.
... iota	Assigns indexes to the consecutive elements of the range.

All aforementioned views can be composed into a pipeline (concatenating using '|'). The pipeline allows several views to be combined simultaneously, increasing the readability and making it more memory efficient (lazy evaluation).

What is a view?

A view is a lightweight range. The key difference between a view and a regular range is that view is only evaluated once it’s actually being used in the code. It doesn’t occupy a space in the memory, unless it is compulsory.

<ranges> usage examples

Filter the vector filled with integers so that only smallest even number is printed.

```
int main() {

    std::vector<int> numbers = { 1, 2, 3, 4 };

    auto even = [](int x) { return x % 2 == 0; };

    auto filtered = numbers | std::views::filter(even) | std::views::take(1);

    for (int x : filtered) {
        std::cout << x << std::endl;
    }

    return 0;
}
```

<ranges> usage examples

Separate a sentence string into smaller word strings.

```
int main() {

    std::string sentence = "This presentation is very interesting";

    auto splitFunction = [](auto&& wordRange) { return std::string(wordRange.begin(), wordRange.end()); };
    auto wordsView = sentence | std::views::split(' ') | std::views::transform(splitFunction);

    for (const auto& word : wordsView) {
        std::cout << word << std::endl;
    };

    return 0;
}
```

Microsoft Visual Studio Debu

```
This
presentation
is
very
interesting
```

<ranges> usage examples

Assign ID numbers to students and print them.

```
int main() {

    std::vector<std::string> students = { "Adam", "John", "Derek" };
    auto updateString = [&](int i) { return std::to_string(i + 1) + ": " + students[i]; };

    auto indexed_view = std::views::iota(0u, students.size()) | std::views::transform(updateString);

    for (std::string entry : indexed_view) {
        std::cout << entry << std::endl;
    }

    return 0;
}
```

Microsoft Visual Studio Debu

```
1: Adam
2: John
3: Derek
```

Transforming a view back into an STL container

```
int main() {

    auto odd = [](int x) { return x % 2 == 1; };

    std::vector<int> vector1 = { 6, 6, 6, 3, 10, 1, 6, 8, 7, 9, 7, 7, 7 };
    auto view = vector1 | std::views::filter(odd);

    // std::ranges::sort(view); - will not work! Instead, do:

    std::vector<int> vector2(view.begin(), view.end());
    std::ranges::sort(vector2);

    return 0;
}
```

After using a pipeline to generate a view, it is not possible to use algorithm operations such as sort - it is obligatory to materialize the view into a regular range.

std::ranges:: ...	
... sort	Reassembles the range into a ascending order, unless otherwise defined.
... for_each	Applies a function to all elements in a range.
... any_of	Returns true if any of the elements in a range fulfill a specified condition.
... all_of	Returns true if all of the elements in a range fulfill a specified condition.
... none_of	Returns true if none of the elements in a range fulfill a specified condition.
... copy	Guess.

Unlike views, those functions cannot be composed in a pipeline.

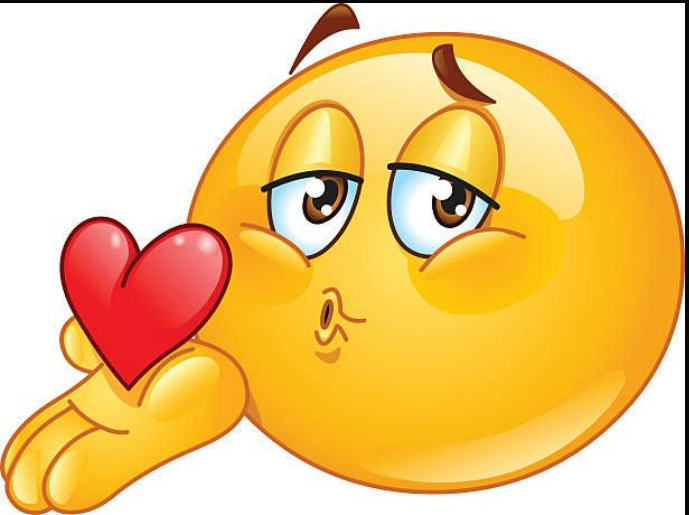
Those are only a select few examples, many more can be found here:

https://en.cppreference.com/w/cpp/ranges.html

https://learn.microsoft.com/en-us/cpp/standard-library/algorithm?view=msvc-170

TASKS AND THE SLIDESHOW:

https://github.com/Rashan1016/SilesianUniversityofTechnology



THANK YOU!